Ingenieria en sistemas de informacion

Optativa Profesional II

Curso Udemy – Flutter

Docente: Mtro. Danilo Rodriguez.

Nombre: Marlon José Suárez Baltodano.

Carné: 2210175

Código Único: 6C492F75

Fecha: 24/04/2026

Sección 2: Instalación. 2
Introducción: 3
Visual studio code, entorno de desarrollo: 3
Instalacion de Flutter en Visual Studio Code. 3
Variable de entorno. 4
Probar variable de entorno. 6
Extensión de Visual Studio Code. 6
Actualización del SDK de Flutter. 8
Dispositivos virtuales. 8
Actualización de Dart, terminal. 9
Sección 3: Dart, nociones básicas. 10
Primer proyecto en Dart. 11
Comentarios. 12
Variables, básico. 13
Operadores. 13
Funciones. 14
Sección 4: Flutter. 14
Aplicación inicial. 15
Introducción a los widgets. 16
Material App. 17
Scaffold. 18
Columnas y Filas. 19
Texto. 20
Contenedor. 21

Sección 2: Instalación.

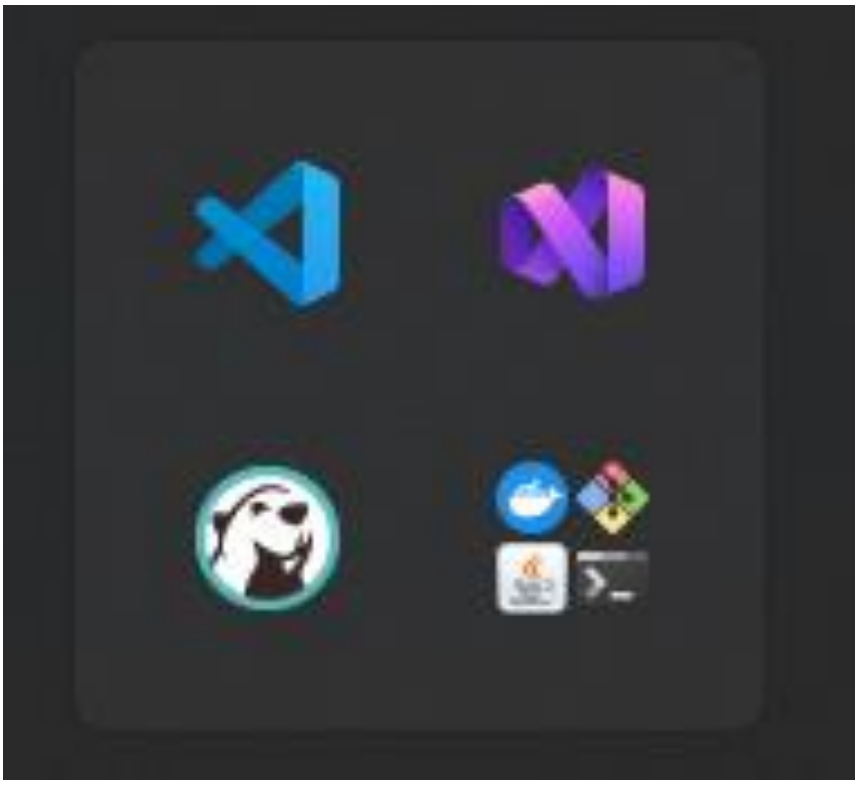
Introducción:

En esta sección se verá cómo instalar las herramientas necesarias para el correcto funcionamiento y desarrollo de las aplicaciones a realizar.

- **Instalación de Visual Studio Code:** editor de código fuente desarrollado por Microsoft para Windows, Linux y macOS.
- **Instalación de Flutter en Visual Studio Code**: Flutter es un SDK de código fuente abierto de desarrollo de aplicaciones móviles creado por Google
- **Creación de dispositivos virtuales:** dispositivos virtuales que permiten emular el comportamiento de un dispositivo físico. Utilizado para poder ver y testear la funcionalidad de la aplicación.
- **Dart upgrade:** actualización del sdk de Dart.
- **Dart upgrade comando**: actualización del sdk de Dart mediante un comando.

Visual studio code, entorno de desarrollo:

En mi caso ya tenía instalado el vs code:



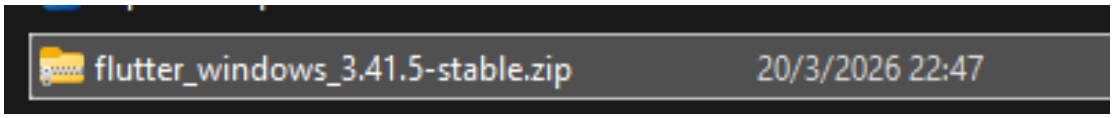
Instalacion de Flutter en Visual Studio Code.

En mi caso ya había instalado el sdk de flutter en clase así que no seguí los pasos tal cual del tutorial de Udemy.

Donde descargue el sdk:

- https://docs.flutter.dev/install/manual

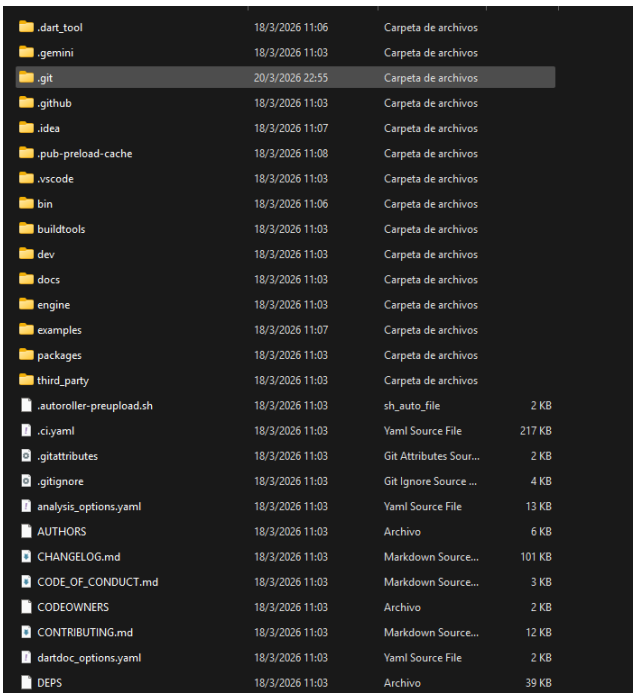
Se va a descargar este archivo:



Este archivo hay que descomprimirlo, hay que crear una carpeta en el disco C:/ en mi caso le puse de nombre flutter a la carpeta y dentro de esta carpeta puse el contenido del sdk que descargué.

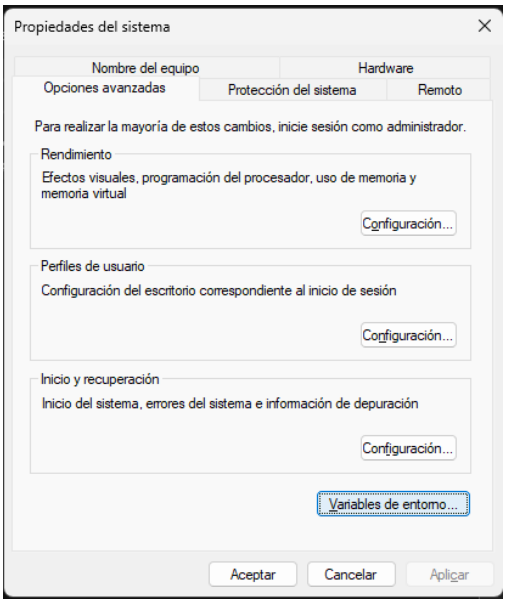
Variable de entorno.

Dentro de la carpeta que creamos el disco C:/ esta el contenido del sdk, vamos a buscar una carpeta llamada “bin”.

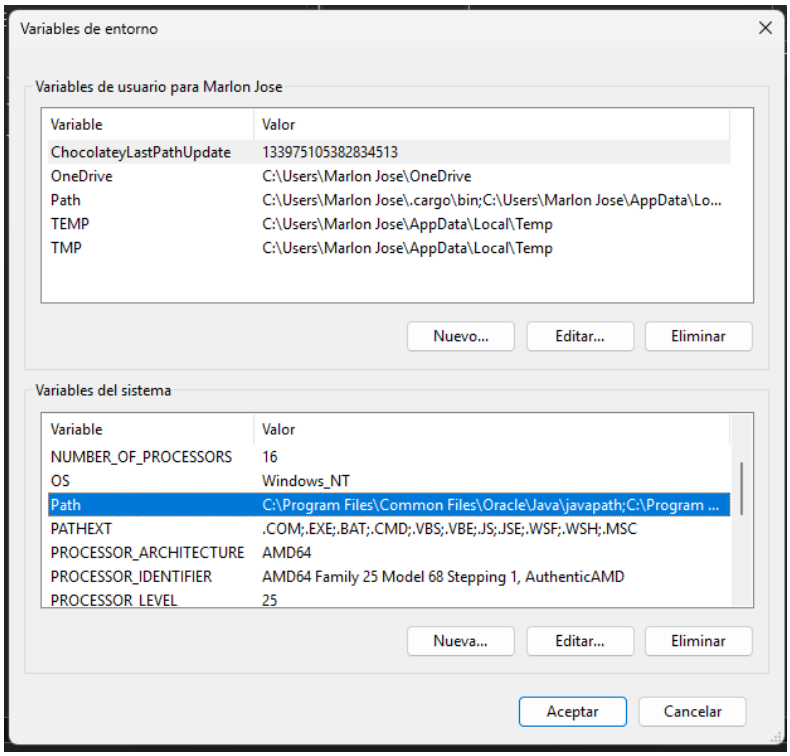


.dart_tool	18/3/2026 11:06	Carpeta de archivos	
.gemini	18/3/2026 11:03	Carpeta de archivos	
.git	20/3/2026 22:55	Carpeta de archivos	
.github	18/3/2026 11:03	Carpeta de archivos	
.idea	18/3/2026 11:07	Carpeta de archivos	
.pub-preload-cache	18/3/2026 11:08	Carpeta de archivos	
.vscode	18/3/2026 11:03	Carpeta de archivos	
bin	18/3/2026 11:06	Carpeta de archivos	
buildtools	18/3/2026 11:03	Carpeta de archivos	
dev	18/3/2026 11:03	Carpeta de archivos	
docs	18/3/2026 11:03	Carpeta de archivos	
engine	18/3/2026 11:03	Carpeta de archivos	
examples	18/3/2026 11:07	Carpeta de archivos	
packages	18/3/2026 11:03	Carpeta de archivos	
third_party	18/3/2026 11:03	Carpeta de archivos	
.autoroller-preupload.sh	18/3/2026 11:03	sh_auto_file	2 KB
.ci.yaml	18/3/2026 11:03	Yaml Source File	217 KB
.gitattributes	18/3/2026 11:03	Git Attributes Sour...	2 KB
.gitignore	18/3/2026 11:03	Git Ignore Source ...	4 KB
analysis_options.yaml	18/3/2026 11:03	Yaml Source File	13 KB
AUTHORS	18/3/2026 11:03	Archivo	6 KB
CHANGELOG.md	18/3/2026 11:03	Markdown Source...	101 KB
CODE_OF_CONDUCT.md	18/3/2026 11:03	Markdown Source...	3 KB
CODEOWNERS	18/3/2026 11:03	Archivo	2 KB
CONTRIBUTING.md	18/3/2026 11:03	Markdown Source...	12 KB
dartdoc_options.yaml	18/3/2026 11:03	Yaml Source File	2 KB
DEPS	18/3/2026 11:03	Archivo	39 KB

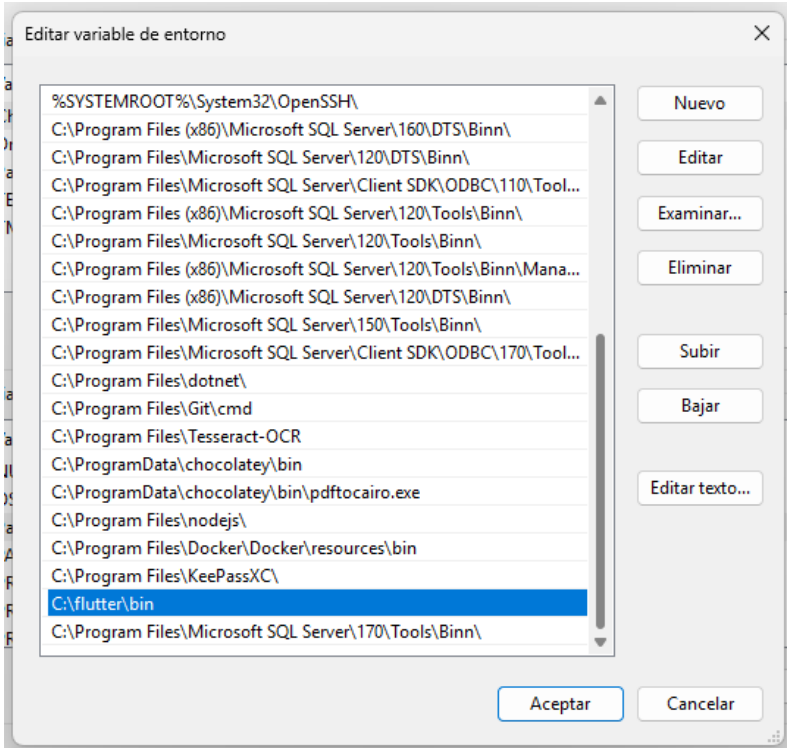
Vamos a copiar la ruta de la carpeta bin, una vez la tengamos copiada buscamos en Windows, buscamos “Editar variables de entorno del sistema” se nos abrira la siguiente ventana y le damos en “Variable de entorno”:



Se nos abrira la siguiente ventana en la que buscaremos en el apartado de “Variables del sistema” la que dice “Path”:

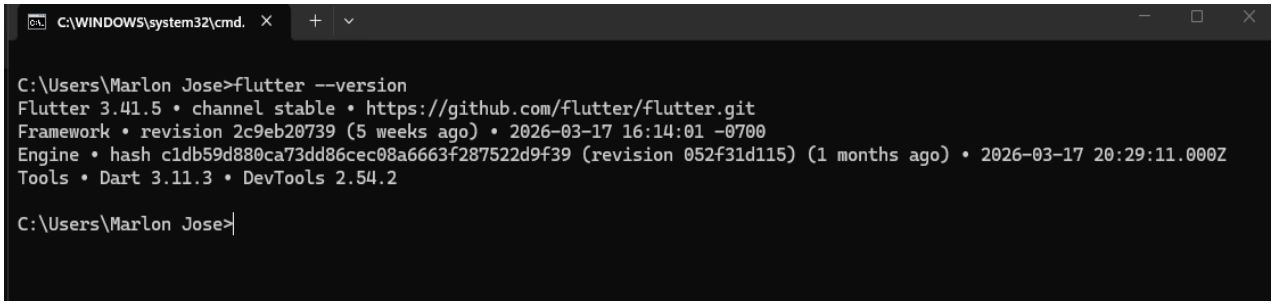


Le damos doble clic y se nos abrirá la siguiente ventana, aquí le daremos en nuevo y pegaremos la ruta de la carpeta bin de flutter, le damos en aceptar a todas las ventanas después para guardar los cambios:



Probar variable de entorno.

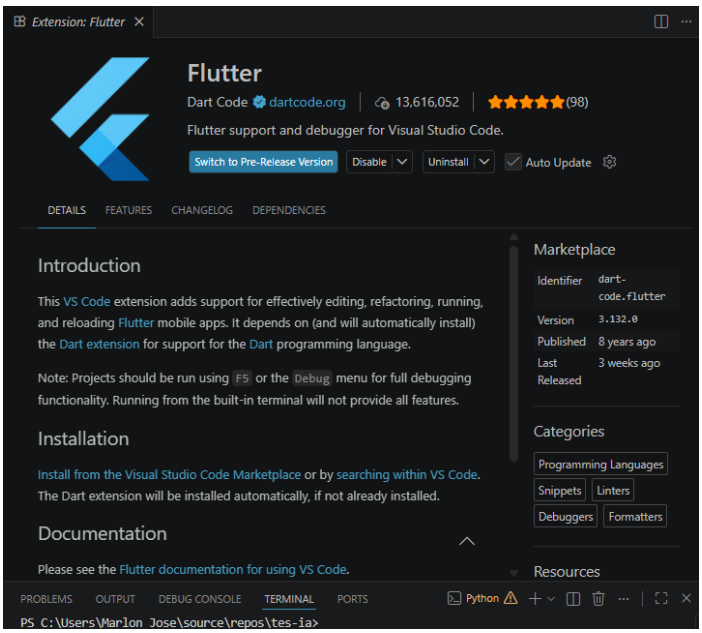
Ahora abrimos la terminar y ejecutamos el siguiente comando para comprobar que se creó y está funcionando la variable de entorno en el sistema:



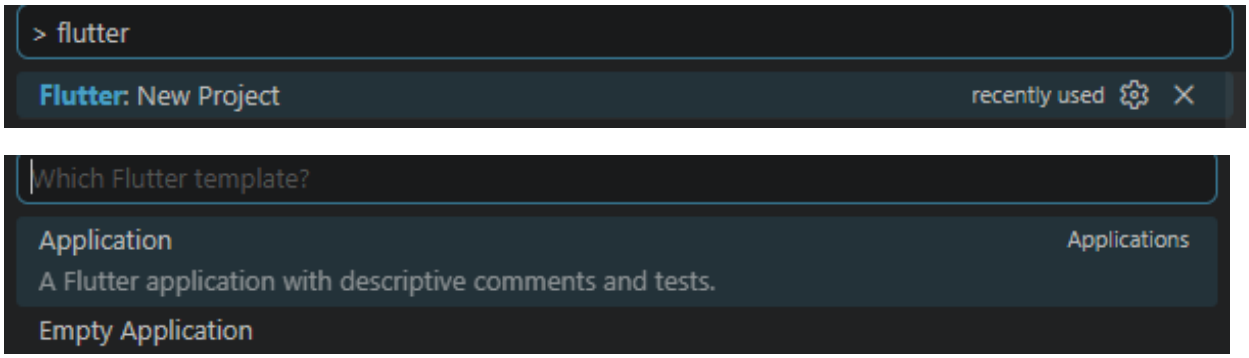
“flutter --version” - Esto nos permitirá ver la versión que tenemos de flutter y probar la variable de entorno.

Extensión de Visual Studio Code.

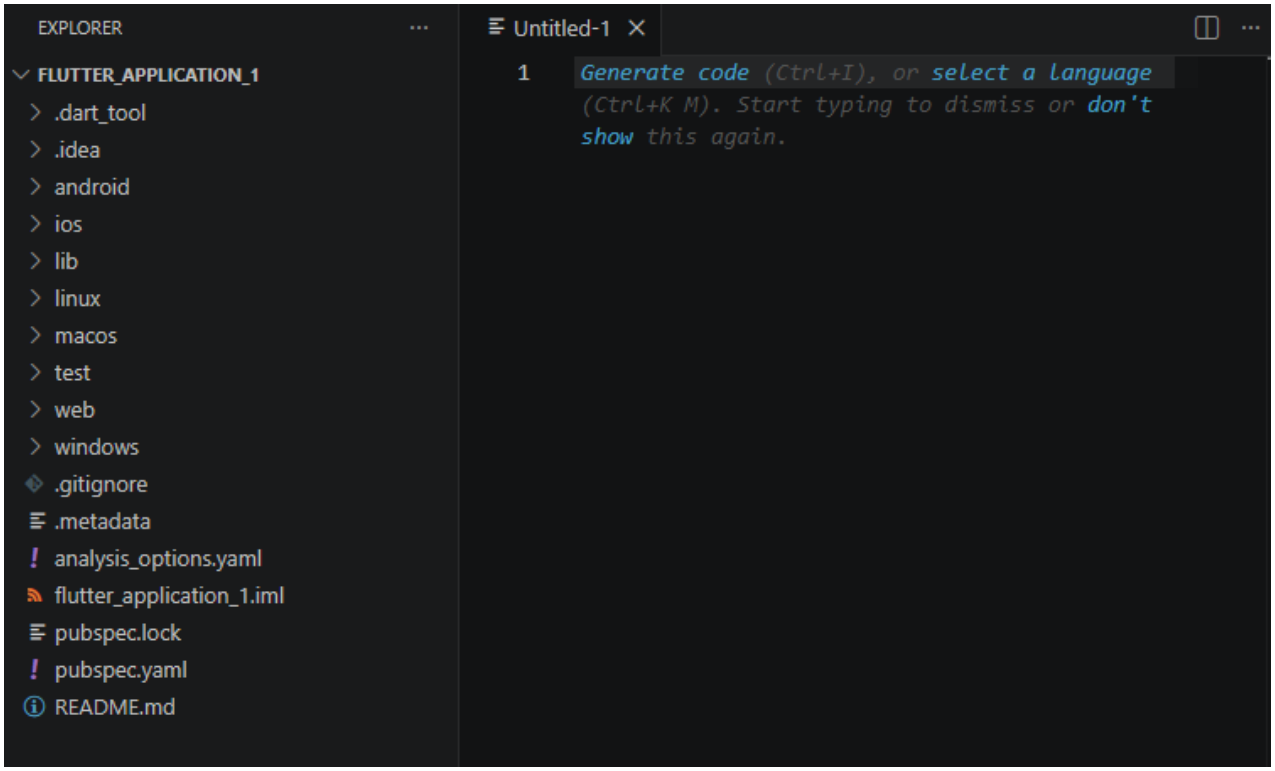
Dentro del vscode en el apartado de extensiones buscamos “flutter” y descargamos la extensión que tiene más descargas.



Ahora vamos al explorador y hacemos [ctrl + shift + P] y buscamos flutter, Seleccionamos nuevo proyecto, aplicacion y luego seleccionamos en que carpeta crear el proyecto y el nombre que le daremos al proyecto:



Y se nos creara el proyecto de esta manera:



Actualización del SDK de Flutter.

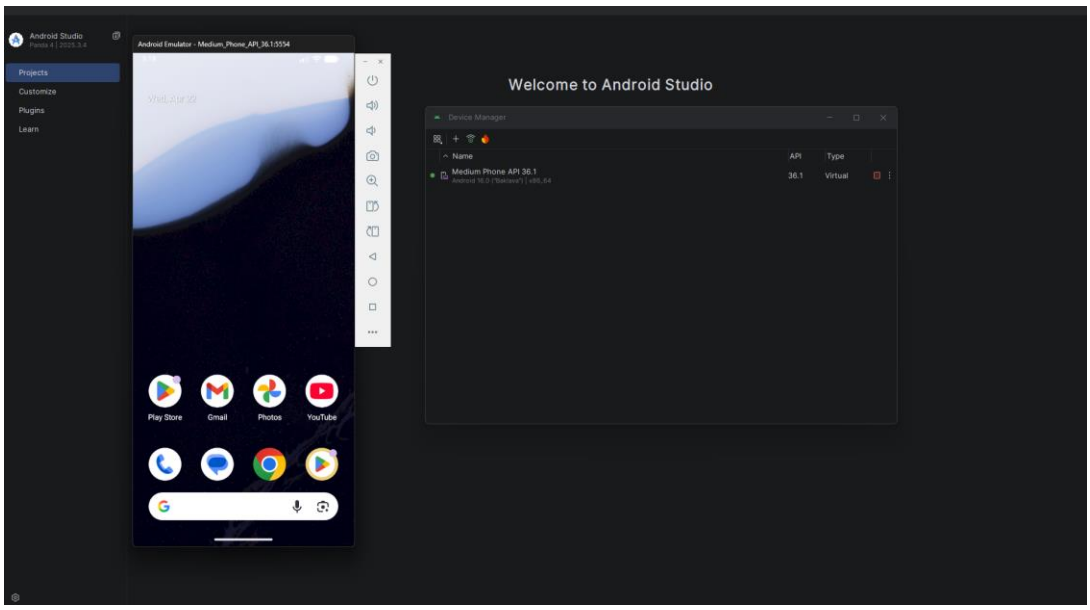
Para actualizarlo abrimos la terminal y ocupamos el comando:

flutter upgrade

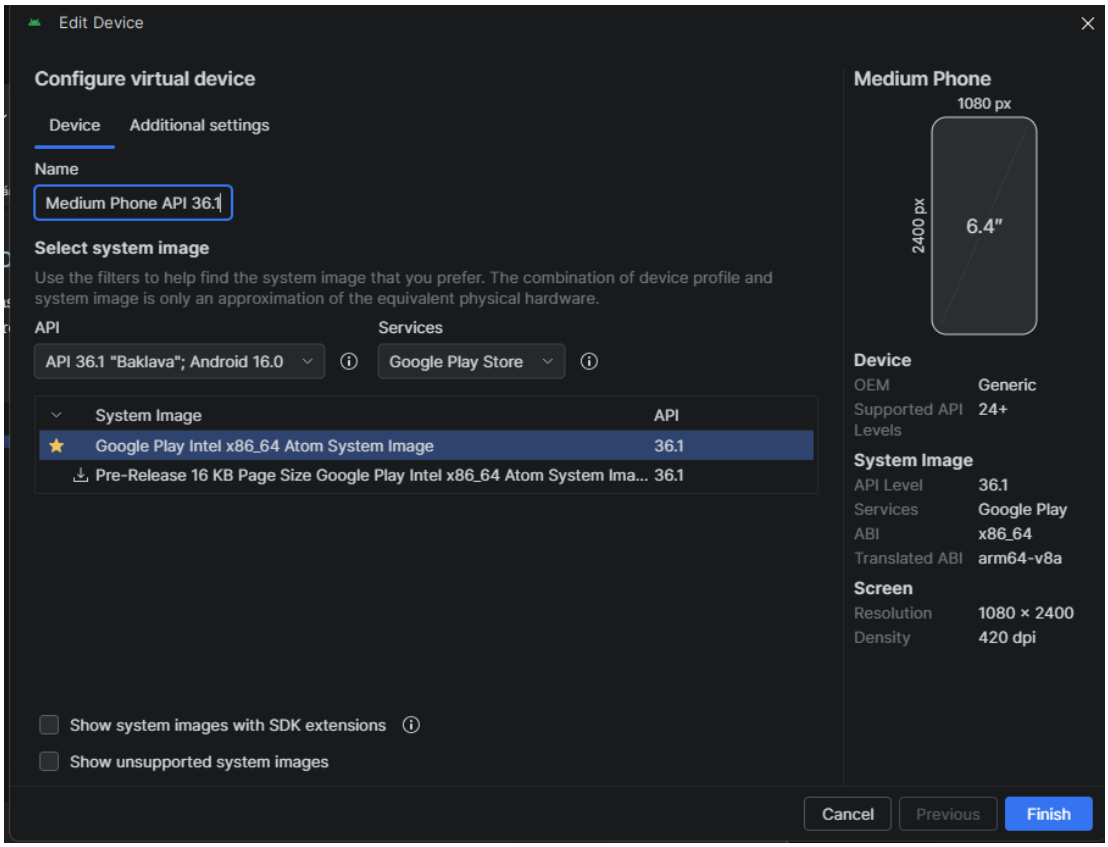
```
C:\Users\Marlon Jose>flutter upgrade
Flutter is already up to date on channel stable
Flutter 3.41.7 • channel stable • https://github.com/flutter/flutter.git
Framework • revision cc0734ac71 (6 days ago) • 2026-04-15 21:21:08 -0700
Engine • hash 7a53c052bc4b472cf780b199087e1368e4a9aa8c (revision 59aa584fdf) (6 days ago) • 2026-04-16 02:32:16.000Z
Tools • Dart 3.11.5 • DevTools 2.54.2
```

Dispositivos virtuales.

En mi caso yo ya tenía instalado el Android studio, también tenía creado un dispositivo virtual creado en clase.



Características:



Actualización de Dart, terminal.

Para actualizar dart desde la terminal, se tiene que encontrar dentro de un proyecto de dart, ya que si se ejecuta directamente desde la terminal el comando no funcionara.

Directamente desde la terminal:

```
C:\WINDOWS\system32\cmd.

Microsoft Windows [Versión 10.0.26200.8246]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Marlon Jose>dart pub upgrade
Found no `pubspec.yaml` file in `C:\Users\Marlon Jose` or parent directories
```

Desde la ruta de un proyecto de Dart:

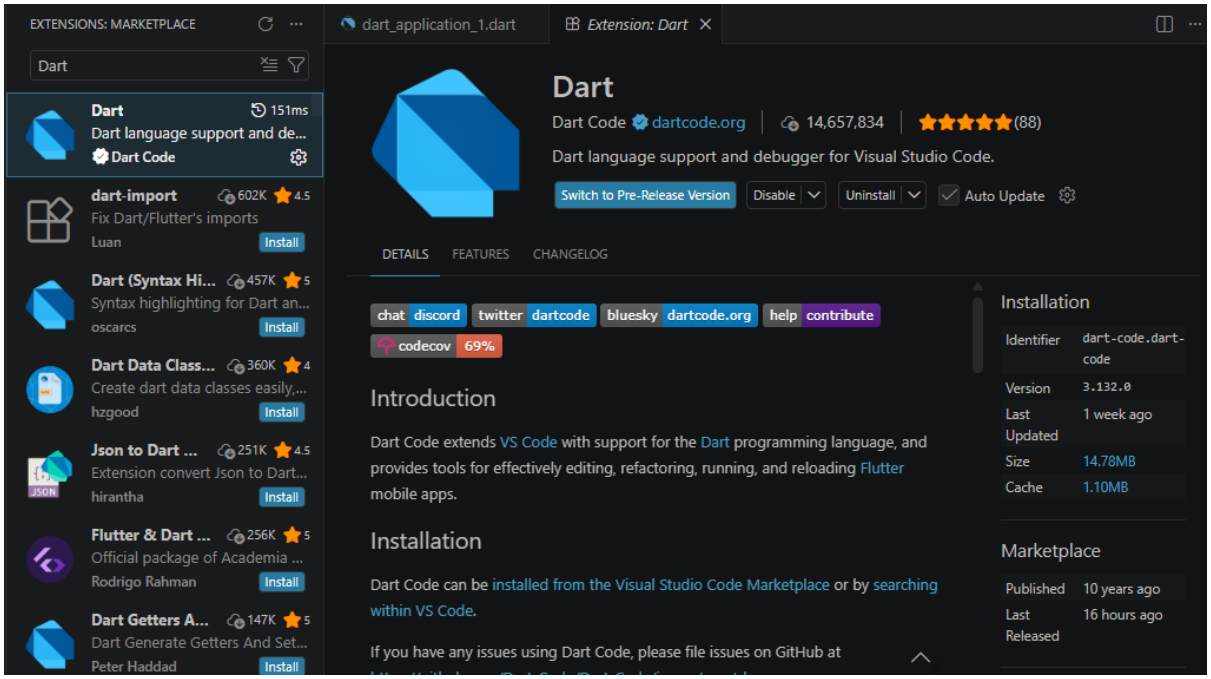
```
EXPLORER
DART_APPLICATION_1
  .dart_tool
  bin
    dart_application_1.dart
  lib
  test
  .gitignore
  analysis_options.yaml
  CHANGELOG.md
  pubspec.lock
  pubspec.yaml
  README.md

dart_application_1.dart   pubspec.yaml
bin > dart_application_1.dart > main

1  import 'dart:io';
2
3  // Revisar pq no da como deberia difetencia de resultado
   Run | Debug
4  void main() {
5  // -- Guardar datos en variables -- //
6    print('Carnet: ');
7    String carnet = stdin.readLineSync()!;
8
9    print('Nombre: ');
10   String nombres = stdin.readLineSync()!;
11
12   print('Apellidos: ');
13   String apellidos = stdin.readLineSync()!;
14
15   print('Cargo: ');
16   String cargo = stdin.readLineSync()!;
17
18   print('Salario: ');
19   double salario = double.parse(stdin.readLineSync()!);
20
21 // -- Calculos de Ley -- //
22 // 1. INSS laboral (7%)
23   double insslab = salario * 0.07;
24 // 2. Base imponible mensual (Salario - INSS)
25   double rentaNetaMensaual = salario - insslab;
26 // 3. Proyecto Anual para el calculo del IR
27   double rentaNetaAnual = rentaNetaMensaual * 12;

PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   powershell

PS D:\Programas Dart\dart_application_1> dart pub upgrade
Resolving dependencies... (2.2s)
Downloading packages...
> vm_service 15.1.0 (was 15.0.2)
Changed 1 dependency!
PS D:\Programas Dart\dart_application_1>
```

Esto solo actualizara las dependencias de este proyecto.

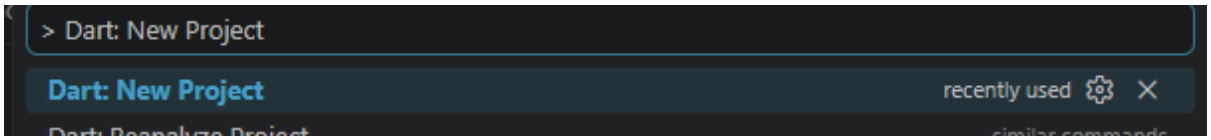
Sección 3: Dart, nociones básicas.

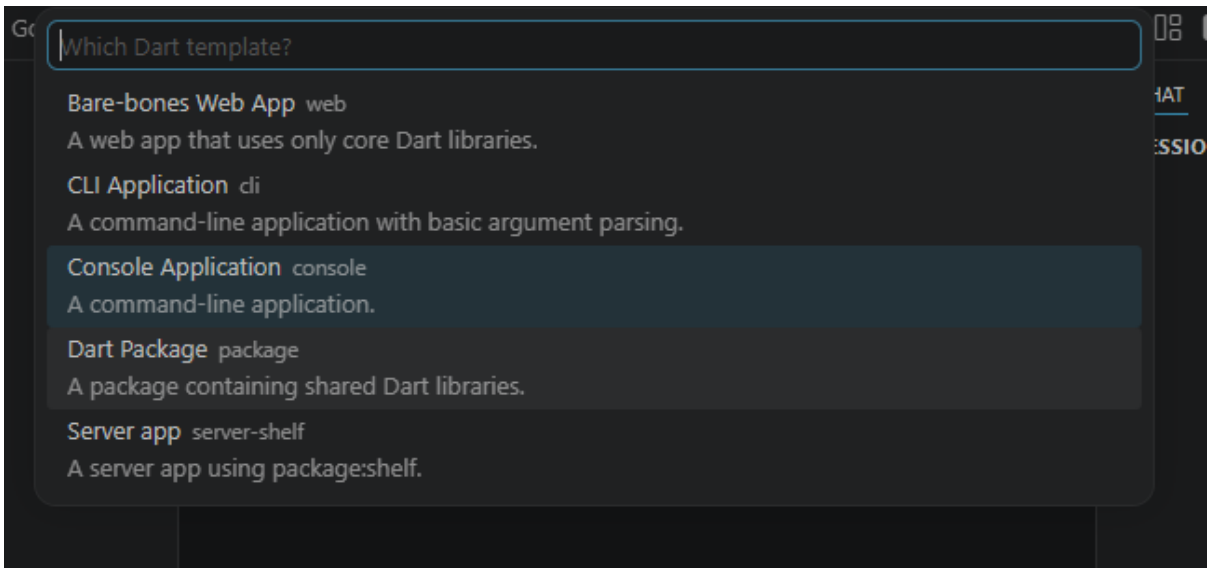
Primer proyecto en Dart.

Se nos pide instalar la siguiente extensión de dart en el vscode:

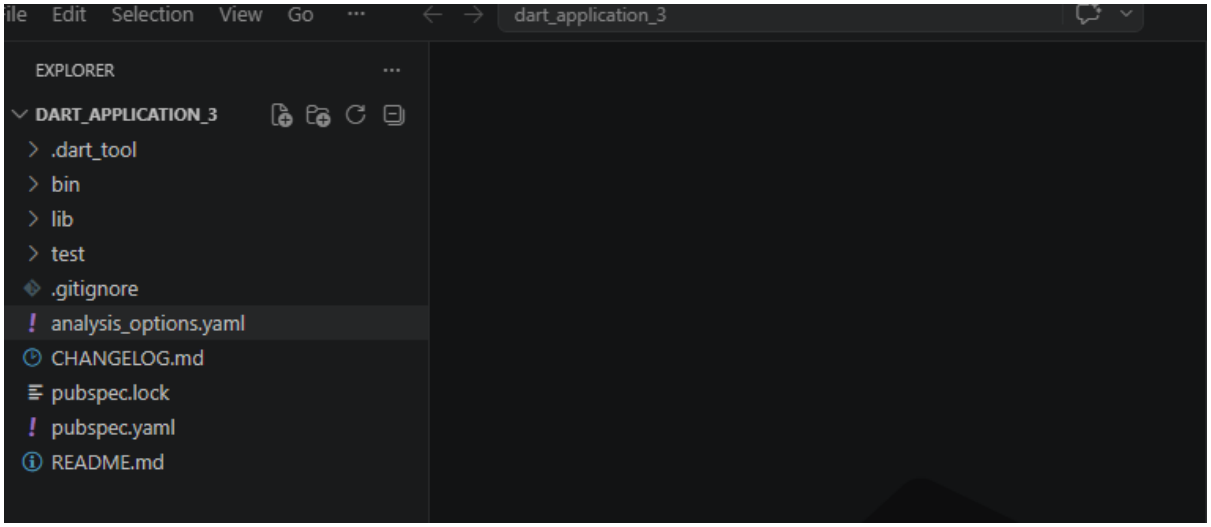


Una vez instalada la extensión le damos en, luego aplicacion de consola:

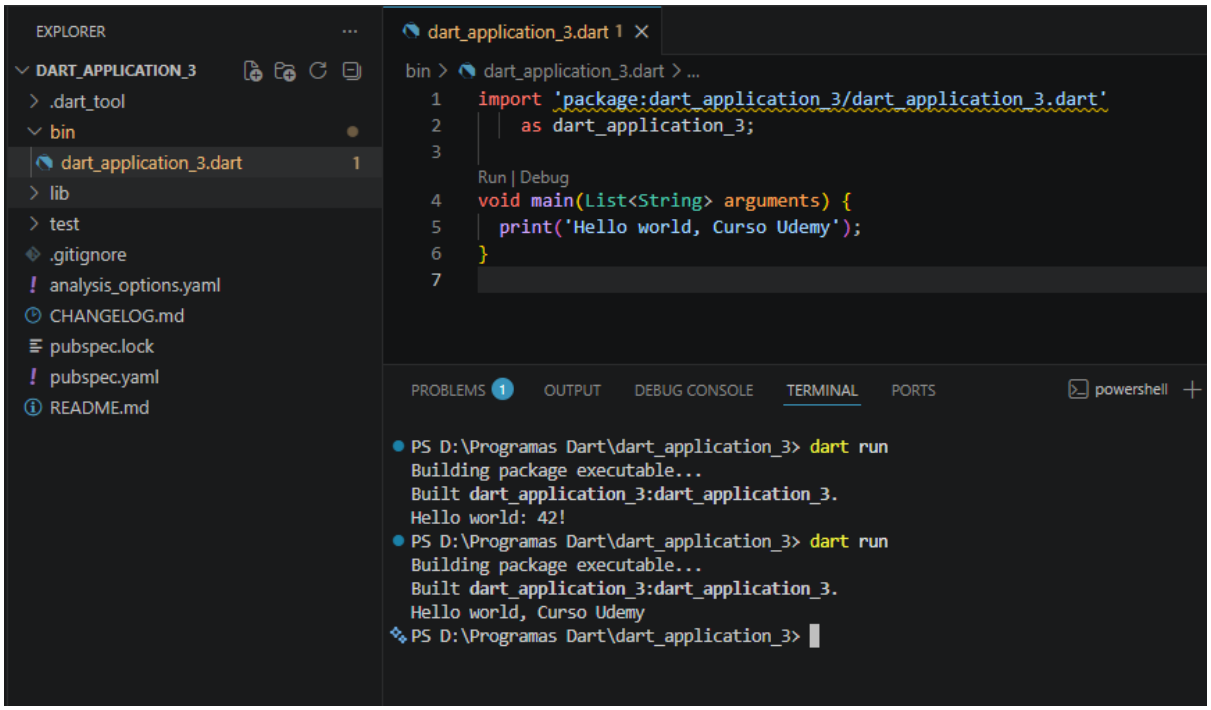




Seleccionamos donde crear el proyecto, luego le damos un nombre al proyecto y se nos creara el siguiente proyecto:



Ahora dentro de la carpeta de “bin” del proyecto modificamos el contenido del archivo “.dart”, una vez modificado guardamos los cambios y lo ejecutamos como en el video o también desde la terminal:



Comentarios.

Se puede comentar una linea con “//” y comentar bloques de codigo con “/**/”

```
dart_application_3.dart 3
bin > dart_application_3.dart > main
2     as dart_application_3;
3
      Run | Debug
4     void main(List<String> arguments) {
5       //print('Hello world, Curso Udemy');
6       /*print('Hola mundo');*/
7       print('<< Hola mundo >>');
8     }
9

PROBLEMS 3   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS D:\Programas Dart\dart_application_3> dart run
Building package executable...
Built dart_application_3:dart_application_3.
<< Hola mundo >>
PS D:\Programas Dart\dart_application_3>
```

Variables, básico.

Ejemplos de variables.

```
dart_application_3.dart 1
bin > dart_application_3.dart > main
3
      Run | Debug
4     void main(List<String> arguments) {
5       //print('Hello world, Curso Udemy');
6       /*print('Hola mundo');*/
7       String variable = 'Hola mundo';
8       int numero = 10;
9       double decimal = 3.14;
10
11      print('Variable: $variable');
12      print('Número: $numero');
13      print('Decimal: $decimal');
14    }
15

PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   powe

PS D:\Programas Dart\dart_application_3> dart run
Building package executable...
Built dart_application_3:dart_application_3.
Variable: Hola mundo
Número: 10
Decimal: 3.14
PS D:\Programas Dart\dart_application_3>
```

Operadores.

```
dart_application_3.dart 1 ×
bin > dart_application_3.dart > main
1   import 'package:dart_application_3/dart_application_3.dart'
2       as dart_application_3;
3
    Run | Debug
4   void main(List<String> arguments) {
5     int a = 10;
6     int b = 20;
7     int sum = a + b;
8     int rest = a - b;
9     int product = a * b;
10    double div = a / b;
11
12    print("The sum of $a and $b is: $sum");
13    print("The rest of $a and $b is: $rest");
14    print("The product of $a and $b is: $product");
15    print("The division of $a and $b is: $div");
16
17    print(1 == 2 || 2 == 1);
18    print(2 == 2 || 2 == 1);
19  }
PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   pc

PS D:\Programas Dart\dart_application_3> dart run
Building package executable...
Built dart_application_3:dart_application_3.
The sum of 10 and 20 is: 30
The rest of 10 and 20 is: -10
The product of 10 and 20 is: 200
The division of 10 and 20 is: 0.5
false
true
PS D:\Programas Dart\dart_application_3>
```

Funciones.

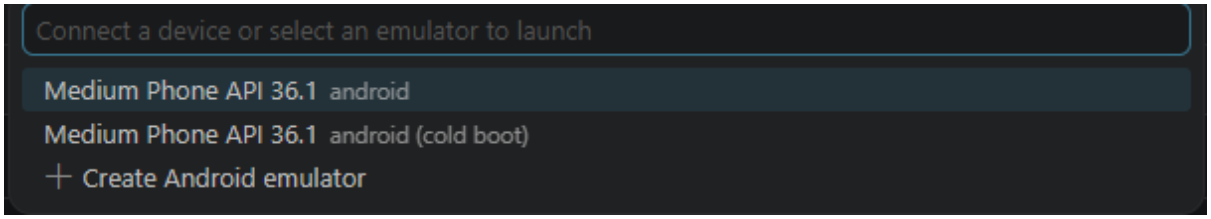
```
dart_application_3.dart 1 ×
bin > dart_application_3.dart > main
1   import 'package:dart_application_3/dart_application_3.dart'
2       as dart_application_3;
3
    Run | Debug
4   void main(List<String> arguments) {
5     printName("Marlon Suarez");
6
7     print(tengoDinero(100.0));
8
9     print(getName());
10  }
11
12  void printName(String name) {
13    print(name);
14  }
15
16  bool tengoDinero(double dinero) {
17    return (dinero > 0);
18  }
19
20  String getName() {
21    var name = "Marlon Suarez";
22    return name;
23  }
24
PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   powershell +

PS D:\Programas Dart\dart_application_3> dart run
Building package executable...
Built dart_application_3:dart_application_3.
Marlon Suarez
true
Marlon Suarez
PS D:\Programas Dart\dart_application_3>
```

Sección 4: Flutter.

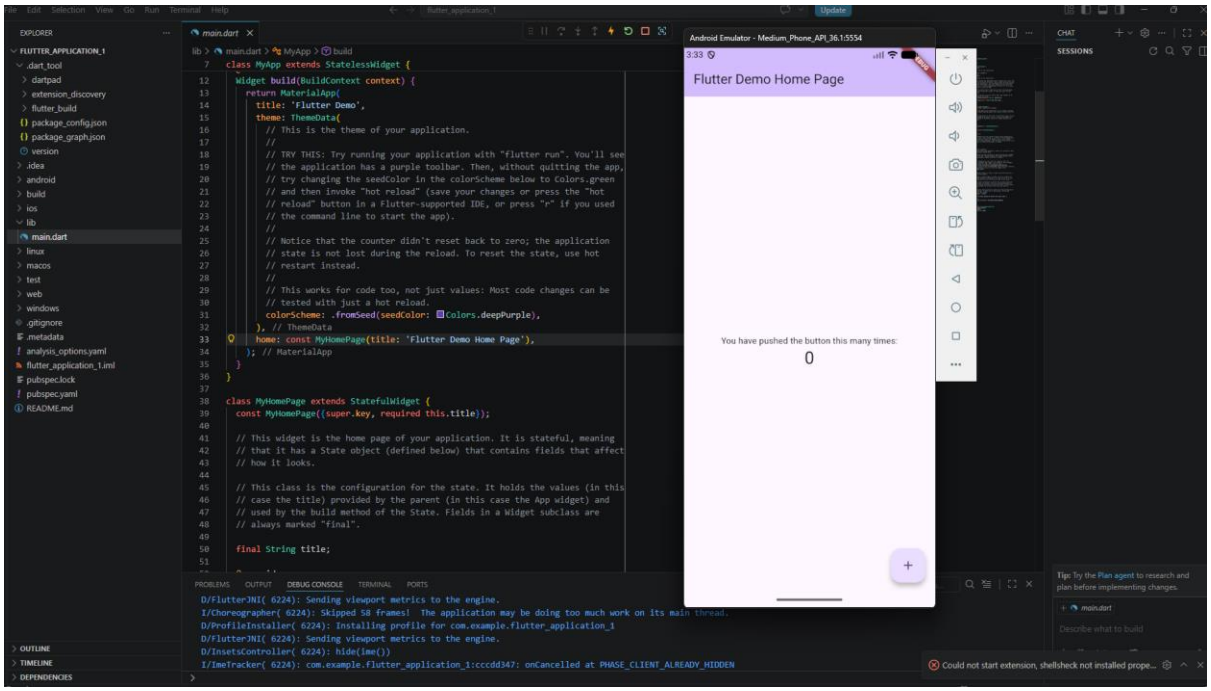
Aplicación inicial.

Creamos el proyecto, abrimos la barra de arriba del vscode como en el video, y escribimos

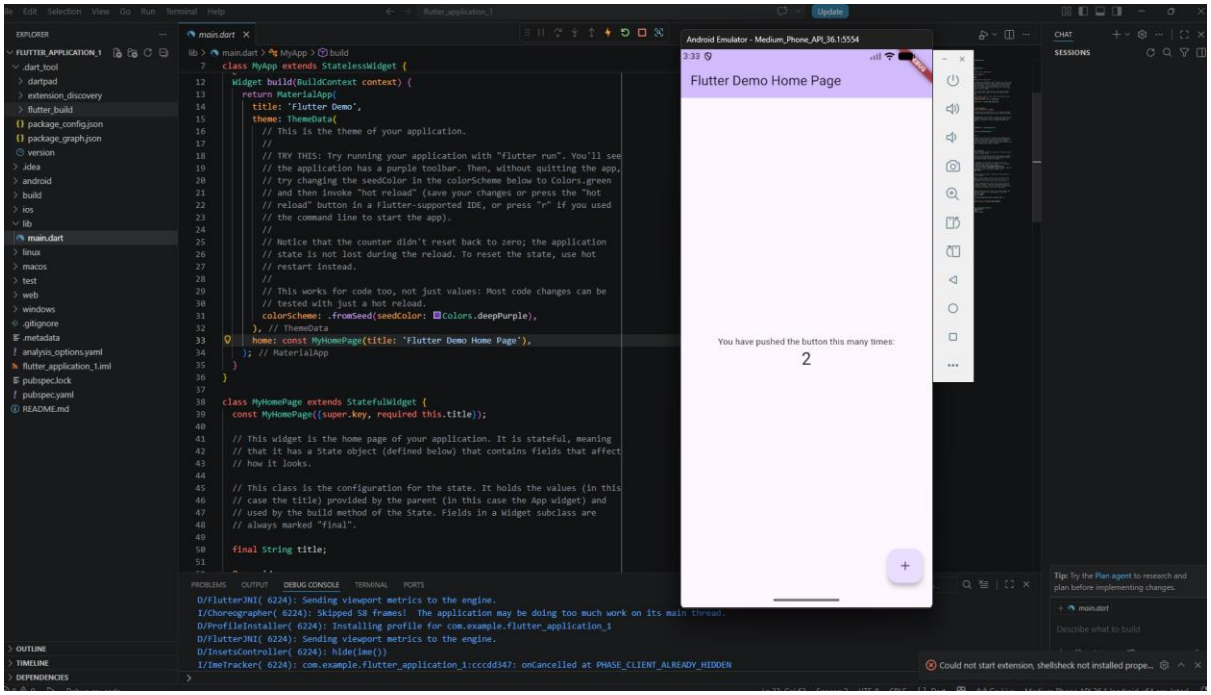
“> flutter: laucher emulator” y a mi me aparecieron estas opciones:



Luego le damos ctrl + f5 para ejecutar el proyecto en el emulador, dilata un poco en ejecutarse, pero se ve así al final:



Probamos la aplicación, al darle al botón de con el símbolo “+” se aumenta el contador del centro de la pantalla.



Introducción a los widgets.

En el video se nos explica el código base del proyecto y se nos explicó que es un widget y esto es un poco de información que encontré de los widgets:

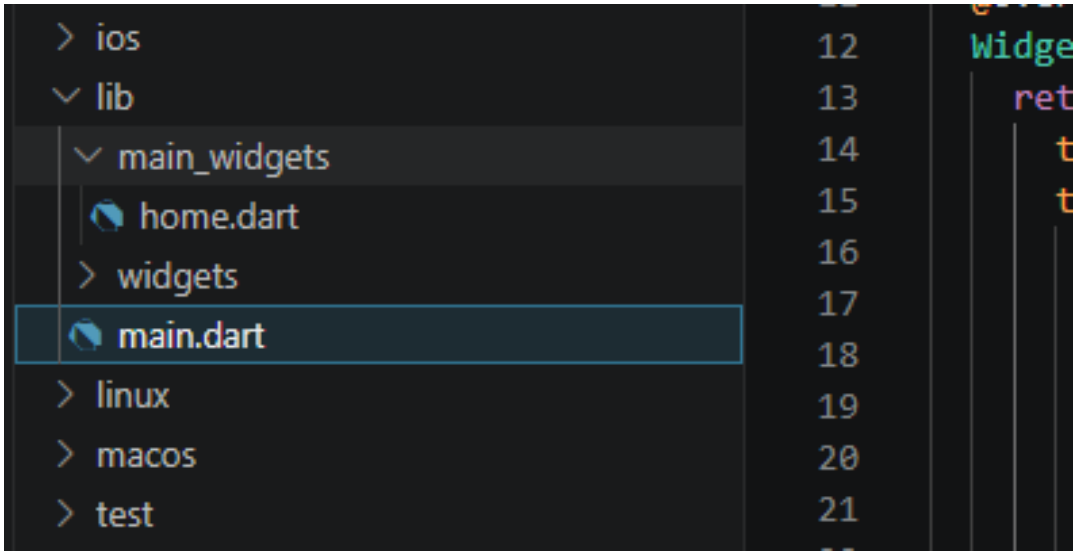
los widgets son los bloques de construcción fundamentales de la interfaz de usuario; todo en Flutter es un widget, desde elementos básicos como texto e iconos hasta estructuras de diseño y la aplicación completa.

Existen dos tipos principales de widgets según su gestión de estado:

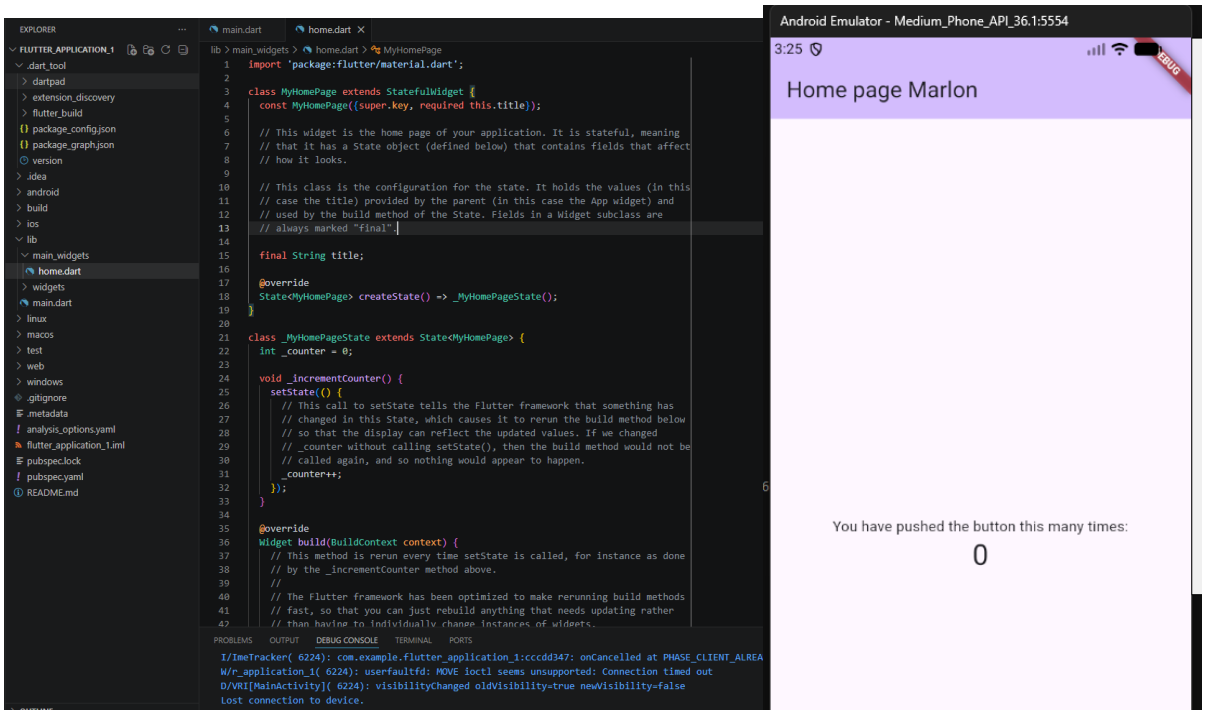
- **StatelessWidget**: Widgets inmutables que no cambian después de ser construidos, ideales para contenido estático como textos, iconos o imágenes fijas.
- **StatefulWidget**: Widgets que pueden almacenar y actualizar su estado interno, permitiendo que la interfaz se reconstruya automáticamente cuando los datos cambian debido a la interacción del usuario o actualizaciones en tiempo real.

Los widgets se categorizan funcionalmente en áreas como **diseño** (Row, Column, Stack), **estructura** (Scaffold, AppBar), **interacción** (Buttons, TextField), **estilo** (Container, Padding) y **animación**. El catálogo oficial de Flutter ofrece una amplia gama de componentes predefinidos para Material Design y Cupertino, permitiendo crear aplicaciones nativas para iOS y Android desde un solo código base.

Tambien creamos dos carpetas dentro de la carpeta “bin” y un archivo home.dart



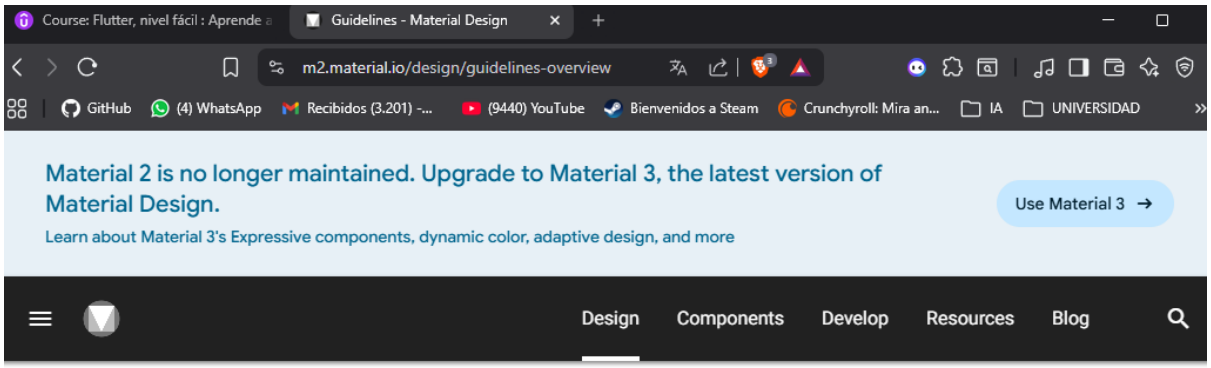
Editamos el main.dart y lo pase al archivo que creamos en main_widgets



Aqui modifique el widget como el tutorial.

Material App.

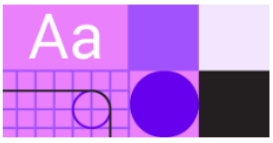
Se nos recomendó ocupar esta pagina, para recomendaciones al hacer diseños para la aplicación.



Create custom Material Themes to make your product unique. Implement your design vision with Material Theming, which simplifies the process of customizing your product and using components, which are the building blocks that make a product usable and functional.

Usability and platform guidance describe how to make sure your app is usable for all users. Platform guidance helps you make detailed decisions about what conventions are proper for each platform.

Up next



Scaffold.

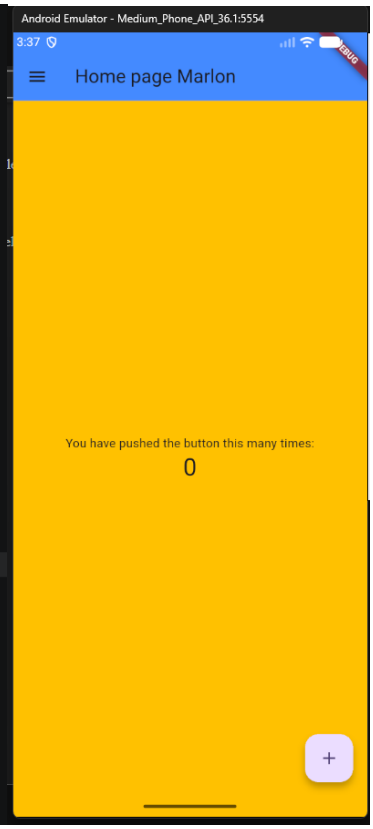
Scaffold es un widget fundamental en Flutter que proporciona la estructura básica de diseño visual de **Material Design** para las pantallas de una aplicación. Actúa como un contenedor principal que organiza y gestiona componentes de interfaz de usuario estándar, asegurando una apariencia coherente y funcional.

Sus características y componentes principales incluyen:

- **AppBar**: Una barra superior para títulos, navegación y acciones.
- **Body**: El área de contenido principal donde se ubican los widgets centrales de la pantalla.
- **FloatingActionButton**: Un botón flotante circular para la acción más prominente de la pantalla.
- **Drawer y EndDrawer**: Paneles deslizantes laterales para navegación.
- **BottomNavigationBar**: Una barra inferior para cambiar entre vistas principales.
- **SnackBar y BottomSheet**: Elementos para mostrar mensajes temporales o información adicional.

En el video nos enseñó a crear un drawer al inicio y al final del sacaffold, modificar el color al fondo del scaffold, también pude modificar el color de la AppBar:

```dart
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.amber, // Cambiar color del fondo scaffold

    appBar: AppBar(
      // backgroundColor: Theme.of(context).colorScheme.inversePrimary,
      backgroundColor: Colors.blueAccent, // Cambiar color de la barra
      title: Text(widget.title),
    ), // AppBar
    body: Center(
      child: Column(
        mainAxisAlignment: .center,
        children: [
          const Text('You have pushed the button this many times:'),
          Text(
            '$_counter',
            style: Theme.of(context).textTheme.headlineMedium,
          ), // Text
        ],
      ), // Column
    ), // Center

    drawer: Drawer(), // Agrega un Drawer al inicio del scaffold

    // endDrawer: Drawer(), // Agrega un endDrawer al final del scaffold
    floatingActionButton: FloatingActionButton(
      onPressed: _incrementCounter,
      tooltip: 'Increment',
      child: const Icon(Icons.add),
    ), // FloatingActionButton
  ); // Scaffold
}
}
```

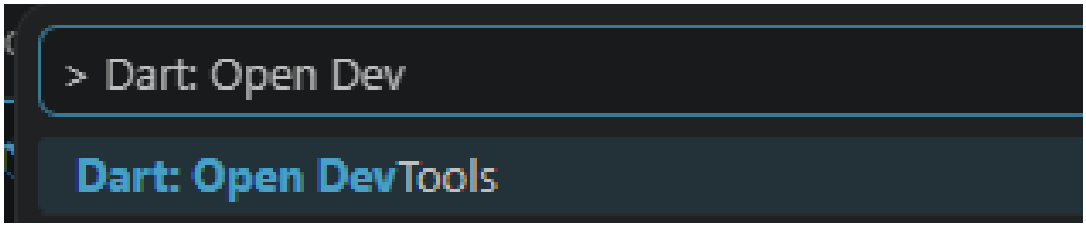


Columnas y Filas.

Las columnas y las filas son los widgets layout más utilizados en flutter ya que pueden tener más de un widget según la estructura que queramos contener.

```
class _MyHomePageState extends State<MyHomePage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // body: Center(child: Text('Hola mundo')));
      body: SafeArea(
        child: Row(
          children: <Widget>[
            Column(
              children: <Widget>[
                Icon(Icons.attachment),
                Icon(Icons.block),
                Icon(Icons.access_alarm),
              ], // <Widget>[]
            ), // Column
            Column(
              children: <Widget>[
                Icon(Icons.account_balance),
                Icon(Icons.account_box),
                Icon(Icons.account_circle),
              ], // <Widget>[]
            ), // Column
          ], // <Widget>[]
        ), // Row
      ), // SafeArea
    ); // Scaffold
  }
}
```

Tambien se nos enseño la siguiente herramienta para que nos ayude con el diseño:

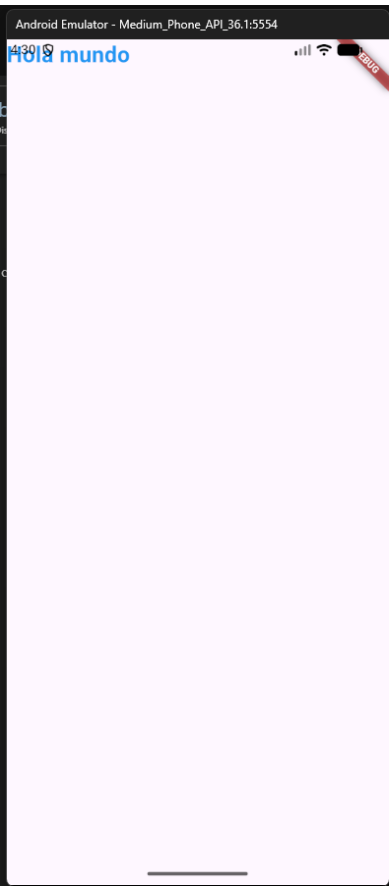


La cual nos da guía de las filas, columnas, el espacio que ocupan dentro de nuestra aplicación.

Texto.

En este apartado mostramos texto en pantalla y aprendimos a alinearlos, darle estilo, color, etc.

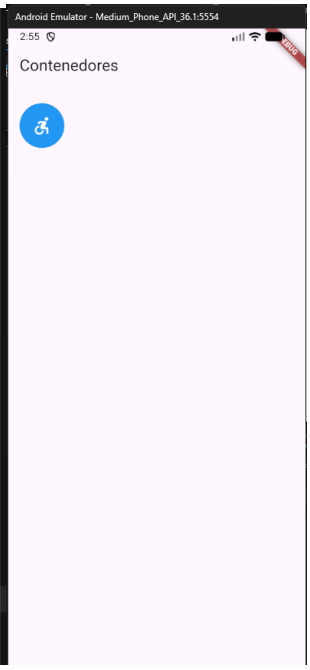
```
// ____Aqui es para la parte de Texto del curso___
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required String title});

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Column(
        children: <Widget>[
          SizedBox(
            width: MediaQuery.of(context).size.width,
            child: Text(
              'Hola mundo',
              style: TextStyle(
                fontWeight: FontWeight.bold,
                fontSize: 24,
                color: Colors.blue,
              ), // TextStyle
            ), // Text
          ), // SizedBox
        ], // <Widget>[]
      ), // Column
    ); // Scaffold
  }
}
```



Contenedor.

En esta parte del video aprendimos a crear contenedores, fragmento de código y resultado:

```
// ____ Aqui en para la parte de contenedores del curso ____
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required String title});

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Contenedores')),

      body: Container(
        margin: EdgeInsets.only(top: 24, left: 16),
        //decoration: BoxDecoration(border: Border.all(color: Colors.black)),
        decoration: BoxDecoration(shape: BoxShape.circle, color: Colors.blue),
        padding: EdgeInsets.all(16),
        child: Icon(Icons.accessible_forward, color: Colors.white, size: 30),
      ), // Container
    ); // Scaffold
  }
}
```



Y con esto finalizamos el curso de flutter.